УНИВЕРСИТЕТ ИТМО

ОТЧЁТ

по лабораторной работе № 2

Использование CouchDB и Etcd в веб-приложении

Вариант № 574821

Работу выполнили:

Самсонова Владислава

Фонарева Виктория

Санкт-Петербург

2026

1. Задача

Доработать приложение первой лабораторной, сохранив существующие HTTP API и key-value-хранилище. Основные объекты необходимо хранить в CouchDB. По варианту вложенные данные представляют историю изменения состояния, связанная коллекция содержит связанные объекты, индексируемый запрос ищет по типу и дате изменения, аналитический запрос возвращает статистику действий пользователей. Изменение структуры добавляет schemaVersion и сведения о последнем изменении.

Приложение сохраняет предметную область первой работы: события библиотеки, временные заявки на посещение, оформленные заказы и настройки менеджеров. Связанными объектами выбраны заказы на событие. В этой версии PostgreSQL заменён CouchDB; Etcd продолжает обслуживать заявки с TTL, кэш, счётчики просмотров и эксперимент конкурентных записей.

2. Соответствие требованиям

Требование	**Реализация**
Постоянные основные объекты	Документы event, order, settings в БД library
Сохранение API первой работы	Прежние контроллеры и JSON-модели; заменён слой хранения
Вложенные данные	Массив history со временем, автором, действием и состоянием
Связанная коллекция	Документы kind=order, ссылка eventId на событие
Фильтры, сортировка, страницы	Mango selector, sort и bookmark
Поиск по вложенным полям	history с оператором $elemMatch
Ревизии и конфликты	_rev, условный PUT, HTTP 409 при устаревшей версии
Два узла	Постоянная односторонняя репликация couch1 → couch2
Аналитический запрос	View user_actions, map по истории, reduce=_sum
Индекс и исследование	Составной JSON-индекс; скрипт сравнения _explain
Валидация	validate_doc_update на уровне CouchDB
Повторяемая миграция	Обновление v1 → v2 с проверкой версии и ревизии
Key-value-кэш	Чтение настроек из CouchDB через Etcd, lease и инвалидация

3. Выбор технологий и архитектура

Используются Java 17, Spring Boot 3.5.16, Gradle, Java HTTP Client, CouchDB 3.5.1, Jetcd 0.8.7 и Etcd 3.6.14. CouchDB предоставляет HTTP/JSON API, ревизии, репликацию, Mango и MapReduce. Отдельная ORM для JSON-документов не требуется: сериализацию выполняет Jackson, а обращения к базе инкапсулирует CouchClient.

```
HTTP / веб-интерфейс
  → Controller → Service → CouchLibraryRepository
                              → CouchClient → couch1
                                                ↓ replication
                                              couch2
                  └→ EtcdGateway → Jetcd → Etcd
```

Основная БД находится на couch1. Второй узел является асинхронной репликой, а не частью единого CouchDB-кластера. Приложение не переключается на него автоматически. Раздельные volumes сохраняют данные двух узлов и Etcd при обычном перезапуске Compose.

В Compose также есть одноразовый сервис setup: он дожидается готовности CouchDB, создаёт базу, design document, индексы и задание репликации. Повторный запуск не создаёт новые ревизии неизменённых design documents и задания репликации.

В первой работе постоянное хранение было отделено от бизнес-сервисов. Это позволило сохранить прежние маршруты, заменить JpaLibraryRepository на CouchLibraryRepository и удалить JPA, Hibernate, PostgreSQL и SQL-схему из рабочего приложения.

4. Представление данных

В CouchDB нет коллекций в смысле MongoDB. Логические коллекции реализованы полем kind и префиксами идентификаторов в одной базе library. Это позволяет реплицировать связанные документы одним заданием и строить общую статистику истории.

Префикс _id	**kind**	**Данные**
event:UUID	event	Название, тип, начало, вместимость, история
order:UUID	order	Заявка, событие, менеджер, количество, статус
settings:managerId	settings	Настройки менеджера, история изменений
_design/library	Служебный	Validation function и аналитическое view

Событие хранит прежние поля id, title, type, startsAt, capacity, createdAt. К ним добавлены _id, _rev, kind, state, schemaVersion, updatedAt, lastModified и history. Пример фрагмента документа после редактирования:

```
{
  "_id": "event:UUID",
  "_rev": "2-opaque-revision",
  "kind": "event",
  "type": "MEETING",
  "schemaVersion": 2,
  "state": "CLOSED",
  "updatedAt": "2026-11-01T12:00:00.000000000Z",
  "lastModified": {
    "by": "alice", "action": "UPDATE", "state": "CLOSED",
    "at": "2026-11-01T12:00:00.000000000Z"
  }
}
```

Это иллюстрация структуры, а не полный допустимый документ. Полный JSON также содержит обязательные бизнес-поля и массив history. Каждая запись истории содержит by, action, state и at; последний элемент совпадает с lastModified. Состояния события: OPEN, CLOSED, CANCELLED, DELETED. Создание старым API записывается от имени system; редактирование передаёт actor.

Время изменения сериализуется в UTC с девятью знаками дробной части. Фиксированный формат необходим для корректного лексикографического сравнения временных строк в Mango. Входные границы from/to нормализуются из ISO-8601 в тот же формат.

5. Связанные объекты и оформление заказа

Заказ содержит eventId. Маршрут GET /api/events/{id}/orders возвращает связанные документы через индекс по kind и eventId; поддерживаются limit и bookmark. Сами события и заказы сохраняются независимо.

Перед созданием заявки и заказа сервис проверяет наличие события. В CouchDB нет внешнего ключа между документами, а validate_doc_update не читает другие документы. Поэтому проверка ссылки и запись заказа не образуют общую атомарную операцию.

Удаление события логическое: состояние становится DELETED, в историю добавляется DELETE. Обычные GET и поиск больше не показывают событие, но его документ остаётся доступен для анализа истории и связей. Заказы не становятся ссылками на физически удалённый документ. Параллельное удаление события и создание заказа всё ещё может привести к заказу на уже логически удалённое событие; это ограничение явно отделено от гарантий базы.

Для новых заказов UUID совпадает с UUID заявки, а _id имеет вид order:UUID. Параллельные попытки оформления создают один и тот же документ: одна запись успешна, другая получает конфликт и читает уже сохранённый заказ. Затем удаляется

временная заявка Etcd. Ошибка очистки не отменяет заказ; повтор запроса или lease удалит остаток.

Между CouchDB и Etcd нет общей транзакции. Если запись в CouchDB не удалась, заявка не удаляется. При потере ответа после успешной записи повтор находит заказ. При переносе старых SQL-заказов сохраняются прежние UUID; дополнительный индекс по draftId позволяет найти такой заказ при повторном оформлении.

6. Фильтрация, сортировка и пагинация

Старый GET /api/events по-прежнему возвращает JSON-массив. Для расширенного поиска введён GET /api/documents/events. Обязательный параметр type задаёт тип события. from и to ограничивают дату последнего изменения включительно. sort принимает asc или desc, limit ограничен диапазоном 1–200.

```
{
  "selector": {
    "kind": "event",
    "type": "MEETING",
    "updatedAt": {"$gte": "2026-01-01T00:00:00.000000000Z"},
    "state": {"$ne": "DELETED"},
    "history": {"$elemMatch": {"by": "alice", "state": "CLOSED"}}
  },
  "sort": [
    {"kind": "asc"}, {"type": "asc"},
    {"updatedAt": "asc"}, {"_id": "asc"}
  ],
  "limit": 25
}
```

$elemMatch гарантирует, что автор и состояние относятся к одной записи истории. Обычная проверка двух независимых полей массива могла бы найти автора в одном элементе, а состояние в другом.

Поля kind и type фиксированы селектором; внутри выбранного типа сортировка идёт по updatedAt, а _id разрешает равенство времён. Для убывания направление меняется у всех полей составной сортировки.

Ответ содержит docs и bookmark. Следующая страница запрашивается с полученным bookmark и теми же фильтрами. Пустой docs означает конец. Bookmark не обеспечивает снимок всей выборки при параллельных изменениях. В отличие от skip, он позволяет продолжать выборку без указания растущего числового смещения.

7. Mango-индекс и _explain

Для запроса по типу и времени создан составной JSON-индекс by-type-updated в design document event-search:

```
{
  "index": {"fields": ["kind", "type", "updatedAt", "_id"]},
  "ddoc": "event-search",
  "name": "by-type-updated",
  "type": "json"
}
```

Порядок полей выбран по запросу: сначала точные равенства kind и type, затем диапазон updatedAt и дополнительный ключ сортировки _id. Фильтр history не является отдельным индексом: он проверяется для кандидатов основной выборки. Поэтому количество просмотренных документов может быть больше числа результатов.

GET /api/documents/events/explain формирует тот же запрос, но отправляет его в _explain. Приложение задаёт use_index; это подсказка планировщику, а не доказательство выбранного индекса. Проверять нужно index.name в плане и предупреждения в ответе _find.

Для исследования реализован scripts/explain_experiment.py. Он создаёт отдельную БД с 500 документами, выполняет одинаковый запрос до и после добавления индекса и проверяет равенство множеств результатов. До создания индекса используется основной _all_docs; после — составной индекс. Сохраняются планы и execution_stats: число просмотренных документов и ключей, количество результатов и время.

Файл фактического протокола: output/evidence/explain.json. Запрос эксперимента не задаёт sort до создания индекса, потому что сортировка по неиндексированным полям может быть отклонена. Это позволяет корректно сравнить одинаковый селектор. Сам endpoint приложения дополнительно проверяет индексированную сортировку и поиск по истории.

8. Аналитика действий пользователей

Design document library содержит view user_actions. Map обходит историю документов и для каждого действия выдаёт единицу с составным ключом [автор, действие]. Reduce использует встроенную функцию _sum.

```
function(doc) {
  if (doc.history && doc.kind) {
    doc.history.forEach(function(entry) {
      if (entry.by && entry.action) {
        emit([entry.by, entry.action], 1);
      }
    });
  }
}
```

Запрос с group=true возвращает число CREATE, UPDATE, DELETE и MIGRATE по каждому автору. Например, два обновления Alice дают ключ ["alice", "UPDATE"] со значением 2. Суммирование ассоциативно, поэтому встроенный reduce корректен и при повторном объединении частичных результатов.

Считаются сохранённые изменения объектов, а не все HTTP-запросы. Чтения, просмотры Etcd и отклонённые попытки записи не входят в статистику. Повторное оформление уже существующего заказа не создаёт новую запись истории. Логическое удаление события сохраняет его вклад в аналитику.

actor и managerId задаются клиентом; аутентификация в учебном приложении отсутствует. Поэтому это статистика заявленных идентификаторов, а не защищённый журнал установленных личностей.

9. Ревизии и конфликт изменений

CouchDB назначает документу _rev. Для изменения существующего документа нужно передать актуальную ревизию. Она является непрозрачным токеном оптимистической блокировки; приложение не вычисляет её самостоятельно.

Сценарий конфликта:

1. Клиенты Alice и Bob читают документ с одной ревизией.
2. Alice отправляет PUT с исходной ревизией; CouchDB сохраняет новую.
3. Bob отправляет PUT со старой ревизией и получает HTTP 409.
4. Bob перечитывает текущий документ, сравнивает изменения и повторяет запрос с актуальной ревизией.

Приложение проверяет rev до изменения и затем передаёт _rev в CouchDB. Проверка базы защищает и от гонки между предварительным чтением и PUT. Ошибки CouchDB 409 преобразуются в ResourceConflictException и HTTP 409 API. Автоматической перезаписи события поверх чужой версии нет.

История в массиве history является прикладной историей. Дерево ревизий CouchDB служит механизмом репликации и конфликтов и не заменяет надёжный архив всех прежних JSON-документов.

10. Репликация между двумя узлами

На couch1 в базе _replicator создаётся документ library-to-replica с source=couch1/library, target=couch2/library, continuous=true. Задание сохраняется в базе и возобновляется после перезапуска. Для доступа используются учебные credentials из Compose.

Репликация переносит документы и design documents вместе с ревизиями. Индексные файлы каждый узел строит локально. Проверка выполняется двумя способами:

чтение состояния задания через _scheduler/docs и чтение события на couch2 до совпадения _rev с couch1. Интеграционный сценарий ограничивает ожидание 40 секундами и завершается ошибкой, если совпадение не достигнуто.

Это асинхронная односторонняя схема. Успешный ответ primary ещё не гарантирует наличия новой версии на replica. Второй узел используется для демонстрации репликации, а не как синхронная резервная копия или автоматический failover.

Приложение пишет только в couch1. Если вручную писать одновременно в оба узла, могут возникнуть конфликтующие ветви дерева ревизий; их ручное объединение не реализовано. Это отличается от локального конфликта устаревшего _rev, который обработан API.

11. validate_doc_update

Функция в couchdb/validate.js проверяет документы на уровне базы, включая записи через прямой HTTP-запрос. Она требует известный kind, schemaVersion=2, корректные поля lastModified, непустую историю и совпадение её последней записи с метаданными изменения.

Для событий проверяются положительная целая вместимость в пределах Java int, название, тип, UUID, корректные даты создания и начала. Для заказов — ссылка на событие, менеджер, количество и идентификаторы. Для настроек — managerId и положительное целое defaultQuantity.

При обновлении запрещено изменять kind, id и createdAt, переписывать прежние элементы истории и уменьшать updatedAt. История должна дополняться. Допускается несколько новых записей за одно полученное обновление, чтобы реплика могла принять состояние, пропустив промежуточные ревизии. API самостоятельно добавляет одну запись на операцию.

Физическое удаление обычных документов запрещено, используется состояние DELETED. Design documents не проверяются как бизнес-объекты. Функция не является полноценной JSON Schema и не реализует внешние ключи. Прямой некорректный PUT получает HTTP 403 от CouchDB; API преобразует отклонение документа в 400.

12. Повторяемая миграция

Миграция реализована в scripts/couch_admin.py migrate. Документы читаются через _all_docs постранично, по 200. Служебные документы пропускаются. Документ без schemaVersion считается версией 1.

Для версии 1 добавляются schemaVersion=2, updatedAt, lastModified и state; история дополняется записью MIGRATE от имени migration. Время берётся из прежнего updatedAt, затем createdAt, а при отсутствии обоих используется начало Unix-эпохи. Бизнес-поля и идентификаторы сохраняются.

PUT выполняется с прочитанной _rev. Конкурентное обновление приводит к 409: скрипт учитывает конфликт и завершается с ненулевым кодом, предлагая повтор. Документы версии 2 и выше не перезаписываются. Поэтому второй запуск не меняет ревизии и не добавляет повторные записи MIGRATE.

Для проверки scripts/migration_experiment.py создаёт отдельную БД, записывает документ старой структуры, устанавливает validation function и запускает миграцию дважды. После первого запуска версия должна стать 2; после второго полный JSON и _rev должны остаться прежними. Фактический протокол сохраняется в output/evidence/migration.json.

Для переноса данных первой лабораторной есть scripts/export_lab1.sql и команда import. SQL формирует JSON с событиями, заказами и настройками. Импорт сохраняет UUID и связи, сразу приводит документы к версии 2 и не перезаписывает существующие _id. Это разовый перенос, а не двусторонняя синхронизация PostgreSQL и CouchDB. Исходная база и Etcd первой работы не удаляются.

13. Etcd: TTL, кэш и инвалидация

Ключ	Назначение	Срок жизни
/library/drafts/{id}	Временная заявка	Lease, 5–86400 секунд
/library/cache/settings/{managerId}	Результат чтения настроек CouchDB	Lease, по умолчанию 60 секунд
/library/views/{eventId}	Счётчик просмотров	Без lease
/library/experiments/conflict	Эксперимент CAS	До перезаписи

Кэш использует стратегию cache-aside. Сервис сначала читает ключ Etcd. При промахе выполняет запрос документа настроек к CouchDB и сохраняет результат с lease. Изменение настроек сначала фиксируется в CouchDB, затем удаляет ключ кэша. Следующее чтение загружает новую версию.

При недоступности Etcd чтение настроек обращается к CouchDB; отказ заполнения необязательного кэша не мешает вернуть результат. При ошибке инвалидации или гонке чтения и изменения возможна временно устаревшая копия. TTL ограничивает время жизни копии с момента помещения в кэш, а не обеспечивает строгую согласованность.

Прямое изменение документа через CouchDB обходит сервисную инвалидацию; кэш обновится после TTL. Это используется в демонстрации истечения кэша. Отрицательные результаты не кэшируются.

Атомарный счётчик просмотров и эксперимент первой работы сохранены: Jetcd сравнивает mod_revision перед записью. Их ревизии относятся к ключам Etcd и не связаны с _rev документов CouchDB.

14. Запуск и проверка

```
podman compose up --build -d
./gradlew test bootJar
node scripts/test_database_rules.cjs
python3 scripts/test_migration.py
python3 scripts/integration.py
python3 scripts/explain_experiment.py
python3 scripts/migration_experiment.py
python3 scripts/pagination_experiment.py
python3 scripts/cache_experiment.py
```

HTTP доступен на localhost:18080, CouchDB на 5984 и 5985, Etcd на 2379. Fauxton открывается по пути /_utils/; учебные логин и пароль — library.

Java-тесты проверяют прежние сценарии событий, заявок, заказов, кэша и CAS, новую историю, конфликты ревизий, построение Mango-запроса и преобразование HTTP-ошибок CouchDB. Сервисные тесты используют in-memory doubles; отдельные тесты CouchClient обращаются к локальному HTTP-серверу. Они не подменяют интеграционную проверку настоящей базы.

JavaScript-проверки выполняют реальную validation function и map-функцию в Node.js. Python-тесты проверяют идемпотентность миграции и нормализацию времени. Интеграционный сценарий обращается к приложению, двум CouchDB и Etcd: проверяет CRUD, историю, HTTP 409, поиск, связанные заказы, analytics, отклонение некорректного документа, репликацию и lease.

Фактические результаты выполнения приведены в следующем разделе. Они отделены от описания ожидаемого поведения.

15. Результаты экспериментов

Проверки выполнены 1 ноября 2026 года. Java: 25 тестов, 0 ошибок и 0 падений; test и bootJar завершились успешно. Дополнительно прошли 13 проверок JavaScript-функций и 2 Python-теста миграции. Сценарии реальных CouchDB 3.5.1 и Etcd 3.6.14 выполнены через локальный API приложения. Контейнер приложения также успешно собран с Java 17.

Индексируемый запрос

Исследование выполнено на 500 документах. Одинаковый селектор вернул 56 документов до и после создания индекса.

Показатель	До индекса	После индекса
Выбранный индекс	_all_docs	by-type-updated
Просмотрено документов	500	56
Просмотрено ключей	500	56
Возвращено документов	56	56
Время выполнения, мс	14,592	25,064

Индекс сократил число просмотренных документов с 500 до 56, примерно в 8,9 раза. Ускорение по времени в этом запуске не получено: первый индексированный запрос занял больше времени. По одному короткому запуску нельзя оценить устойчивую задержку; построение и прогрев индекса, кэши и планирование процессов влияют на результат. Измерение подтверждает выбор индекса и уменьшение объёма просмотра, но не доказывает выигрыш времени.

Миграция, репликация и API

Первый запуск миграции обновил один старый документ, второй пропустил его: updated=0, skipped=1. Полный JSON и ревизия после второго запуска совпали с результатом первого. Конфликтов при этом опыте не было.

Интеграционный сценарий подтвердил сохранение и чтение события, добавление истории, HTTP 409 при повторе старой ревизии, поиск по автору вложенного изменения, получение связанного заказа и идемпотентное оформление. View вернул действия Alice; прямой PUT с capacity=0 был отклонён CouchDB с HTTP 403. Документ на втором узле получил ту же ревизию, что и обновлённый документ на первом. Заявка с TTL 5 секунд исчезла из Etcd, а API вернул 404. Логическое удаление события также привело к 404 обычного GET.

Отдельный опыт создал три события одного типа. Чтение по одной записи через bookmark вернуло каждое событие ровно один раз; четвёртая страница оказалась пустой. Убывающая сортировка дала обратный порядок.

Кэширование

После сохранения настроек со значением 3 первое чтение заполнило кэш. Изменение через сервис на 4 инвалидировало кэш, и следующее чтение вернуло 4. Затем документ CouchDB был напрямую изменён на 7, в обход инвалидации: API ещё возвращал кэшированное 4. Через 60,79 секунды наблюдения lease истёк, и API загрузил 7 из CouchDB. Настроенный TTL равен 60 секундам; небольшое превышение связано с дискретным опросом и фактическим истечением lease.

Протоколы: integration.json, explain.json, migration.json, pagination.json и cache.json в каталоге output/evidence. Исходные измерения сохранены отдельно от текста отчёта,

поэтому результаты можно перепроверить или получить заново скриптами из README.

16. Ограничения и вывод

CouchDB отвечает за постоянные документы, ревизии, индексированные запросы, аналитику и репликацию. Etcd сохранён для временных данных, кэша и атомарных операций. Вложенная история позволяет искать изменения и агрегировать действия пользователей без отдельной таблицы журнала.

Основные ограничения: отсутствие междокументных транзакций и внешних ключей, асинхронная репликация, отсутствие автоматического failover и объединения конфликтующих ветвей, нестрогая актуальность кэша, отсутствие аутентификации и проверки оставшихся мест. История растёт вместе с документом; для большого журнала потребовались бы отдельные документы действий и политика хранения.

Сохранение старых HTTP-моделей отделено от новой внутренней структуры. Повторяемая миграция и проверка _rev позволяют изменять формат данных без молчаливого перезаписывания конкурентных изменений.

Источники

- CouchDB: Mango, _find и _explain.
- CouchDB: design documents, MapReduce и validation.
- CouchDB: постоянная репликация через _replicator.
- CouchDB: конфликты репликации.
- Etcd: API, lease и условные транзакции.